

UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ
FACULTAD DE ESTUDIOS PROFESIONALES ZONA MEDIA

MATERIA:
Microcontroladores

PRÁCTICA 4:
”Parpadeo del LED integrado a la
RPI PICO W usando C”

Alumno: CESAR GAEL MANCILLA ESTRADA
Matrícula: 382031
Docente: Ing. Jesús Padrón
Fecha de entrega: 25 de febrero de 2026

RIOVERDE, B.C., 25 de febrero de 2026

Índice

1. Introducción	2
2. Desarrollo	2
2.1. Configuración del Entorno de Desarrollo	2
2.1.1. Instalación y configuración del SDK de Raspberry Pi	2
2.1.2. Descripción de las librerías utilizadas	2
2.1.3. Creación de la estructura de carpetas	3
2.2. Creación del Proyecto	3
2.2.1. Uso de Pico - Developer PowerShell	3
2.2.2. Archivo CMakeLists.txt	3
2.2.3. Implementación del código en C	4
2.3. Proceso de Compilación y Carga del Programa	6
2.3.1. Compilación en Visual Studio Code	6
2.3.2. Generación del archivo .uf2	6
2.3.3. Transferencia a la Raspberry Pi Pico W	7
2.3.4. Verificación del funcionamiento	7
3. Resultados	8
3.1. Código utilizado	8
3.2. Evidencias del proceso	8
3.3. Errores encontrados y soluciones	9
4. Conclusión	9
4.1. Aprendizaje adquirido	9
4.2. Posibles mejoras en la implementación	10
4.3. Aplicaciones prácticas del conocimiento adquirido	10
Anexo: Criterios de Evaluación	11

1. Introducción

La presente práctica tiene como objetivo principal aprender a crear un proyecto desde cero para programar la Raspberry Pi Pico W, utilizando como referencia el código de ejemplo que permite el parpadeo del LED integrado en la placa. Este ejercicio fundamental permite comprender la estructura básica de un proyecto para microcontroladores basados en el SDK de Raspberry Pi.

El uso del SDK de Raspberry Pi es de vital importancia ya que proporciona las librerías y herramientas necesarias para programar el microcontrolador en lenguaje C sin necesidad de partir desde cero. El SDK incluye funciones de alto nivel que abstraen la complejidad del hardware, permitiendo al desarrollador concentrarse en la lógica de su aplicación.

La Raspberry Pi Pico W, equipada con el microcontrolador RP2040 y conectividad inalámbrica, tiene numerosas aplicaciones en el mundo de los microcontroladores: sistemas embebidos, Internet de las Cosas (IoT), domótica, robótica educativa, wearables y prototipado rápido, entre otros. Su bajo costo, bajo consumo de energía y versatilidad la convierten en una plataforma ideal para el aprendizaje y desarrollo de proyectos profesionales.

2. Desarrollo

2.1. Configuración del Entorno de Desarrollo

2.1.1. Instalación y configuración del SDK de Raspberry Pi

Para el desarrollo de esta práctica, se instaló el SDK de Raspberry Pi Pico siguiendo las instrucciones oficiales. Este SDK incluye las toolchains necesarias para compilar código C para la arquitectura ARM del RP2040, así como las librerías fundamentales para la interacción con el hardware.

2.1.2. Descripción de las librerías utilizadas

Las librerías empleadas en esta práctica son:

- **pico_stdlib:** Librería estándar del SDK que incluye funciones básicas para el control de pines GPIO, temporizadores, comunicación UART, entre otras. De esta librería se utiliza la función `sleep_ms()` para generar los retardos en el parpadeo.
- **pico_cyw43_arch_none:** Librería específica para el chip inalámbrico CYW43 de la Pico W. La variante `_none` indica que no se utilizan las funciones de red, solo las de control del GPIO del chip. Esta librería es necesaria para acceder al LED integrado, que está conectado a través de este componente.

2.1.3. Creación de la estructura de carpetas

Se creó una carpeta principal llamada MICROCONTROLADORES que almacenará todas las prácticas del curso. Dentro de esta, se creó la carpeta específica para la práctica 4 con el nombre Práctica_4, respetando la recomendación de no utilizar espacios en blanco.

2.2. Creación del Proyecto

2.2.1. Uso de Pico - Developer PowerShell

Se abrió la aplicación **Pico - Developer PowerShell** con privilegios de administrador y se navegó hasta la carpeta del proyecto utilizando el comando:

```
cd C:\Users\epro5\Documents\MICROCONTROLADORES\Pr ctica_4
```

```

C:\> Copyright (C) Microsoft Corporation. Todos los derechos reservados.
C:\> Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows
C:\> PICO_playground_PATH=C:\Users\epro5\Documents\Pico-v1.5.1\pico-playground
C:\> PICO_repos_PATH=C:\Users\epro5\Documents\Pico-v1.5.1
C:\> PICO_REG_KEY=Software\Raspberry Pi\Pico SDK v1.5.1
C:\> PICO_INSTALL_PATH=C:\Program Files\Raspberry Pi\Pico SDK v1.5.1
C:\> PICO_examples_PATH=C:\Users\epro5\Documents\Pico-v1.5.1\pico-examples
C:\> PICO_SDK_PATH=C:\Program Files\Raspberry Pi\Pico SDK v1.5.1\pico-sdk
C:\> PICO_SDK_VERSION=1.5.1
C:\> PICO_extras_PATH=C:\Users\epro5\Documents\Pico-v1.5.1\pico-extras
C:\> OPENOCD_SCRIPTS=C:\Program Files\Raspberry Pi\Pico SDK v1.5.1\openocd\scripts
PS C:\Program Files\Raspberry Pi\Pico SDK v1.5.1> cmake --version
cmake version 3.25.2

CMake suite maintained and supported by Kitware (kitware.com/cmake).
PS C:\Program Files\Raspberry Pi\Pico SDK v1.5.1> cd "C:\Users\epro5\Documents\MICROCONTROLADORES\Práctica_4"
PS C:\Users\epro5\Documents\MICROCONTROLADORES\Práctica_4> cd build
PS C:\Users\epro5\Documents\MICROCONTROLADORES\Práctica_4\build> cmake --build .
[57/57] Linking CXX executable Parpadeo.elf
PS C:\Users\epro5\Documents\MICROCONTROLADORES\Práctica_4\build>
  
```

Figura 2: Enter Caption

Figura 3: Navegación a la carpeta del proyecto en PowerShell
Espacio para captura de pantalla del PowerShell con el comando cd

2.2.2. Archivo CMakeLists.txt

Se creó el archivo CMakeLists.txt con el siguiente contenido:

```

1 cmake_minimum_required(VERSION 3.13)
2
3 include(pico_sdk_import.cmake)
4 set(PICO_BOARD pico_w)
5
6 project(Pr ctica_4)
7
8 pico_sdk_init()
  
```

```
9
10 add_executable (Parpadeo
11     Parpadeo.c
12 )
13
14 target_link_libraries (Parpadeo
15     pico_stdlib
16     pico_cyw43_arch_none
17 )
18
19 pico_add_extra_outputs (Parpadeo)
```

Este archivo funciona como las instrucciones de construcción para CMake. A continuación se explica su funcionamiento:

- `project (Práctica_4)`: Define el nombre del proyecto.
- `add_executable (Parpadeo Parpadeo.c)`: Indica que se debe crear un ejecutable llamado "Parpadeo" a partir del archivo fuente "Parpadeo.c".
- `target_link_libraries (...)`: Especifica las librerías que deben enlazarse con el ejecutable.
- `pico_add_extra_outputs (Parpadeo)`: Genera archivos adicionales como el .uf2 necesario para programar la placa.

2.2.3. Implementación del código en C

Se creó el archivo `Parpadeo.c` con el código necesario para controlar el LED integrado:

```
1 #include "pico/stdlib.h"
2 #include "pico/cyw43_arch.h"
3
4 int main() {
5     // Inicializar el hardware del chip inalambrico
6     if (cyw43_arch_init()) {
7         return -1;
8     }
9
10    // Bucle infinito de parpadeo
11    while (true) {
12        cyw43_arch_gpio_put (CYW43_WL_GPIO_LED_PIN, 1); // Encender LED
13        sleep_ms (250); // Esperar 250ms
14        cyw43_arch_gpio_put (CYW43_WL_GPIO_LED_PIN, 0); // Apagar LED
```

```
15     sleep_ms(250); // Esperar 250ms
16 }
17 }
```

Explicación del código:

- Las directivas `#include` incorporan las cabeceras de las librerías necesarias.
- `cyw43_arch_init()`: Inicializa el hardware del chip inalámbrico, requerido para acceder al GPIO donde está conectado el LED.
- `while (true)`: Bucle infinito que mantiene el programa en ejecución continua.
- `cyw43_arch_gpio_put()`: Función que escribe un valor digital en un pin del chip CYW43. Se utiliza `CYW43_WL_GPIO_LED_PIN` que es la constante que identifica el pin del LED integrado.
- `sleep_ms()`: Función que genera una pausa en milisegundos.

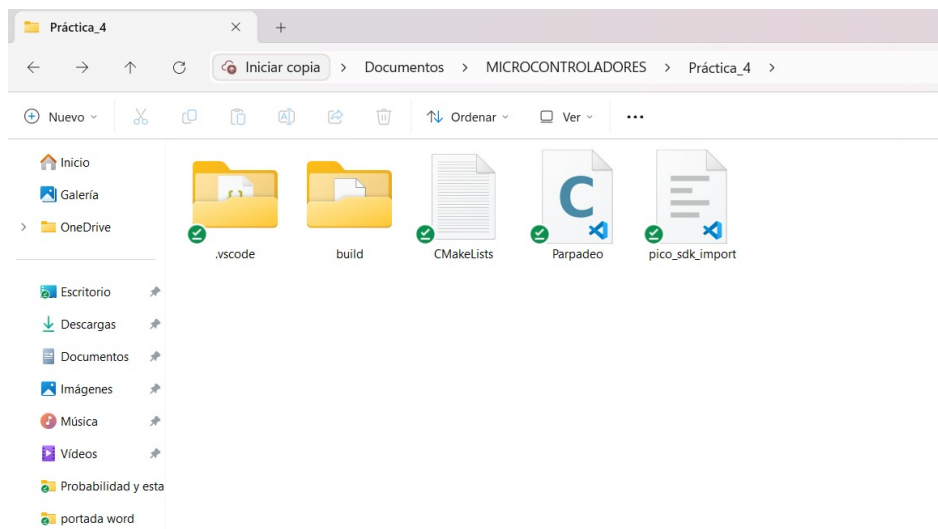


Figura 4: Enter Caption

Figura 5: Archivos del proyecto en el explorador de Windows

Espacio para captura de pantalla de los archivos CMakeLists.txt y Parpadeo.c en la carpeta

2.3. Proceso de Compilación y Carga del Programa

2.3.1. Compilación en Visual Studio Code

Se abrió la aplicación **Pico - Visual Studio Code** y se cargó la carpeta del proyecto a través de **File → Open Folder**. Al abrir la carpeta, VSCode reconoció automáticamente el proyecto CMake y solicitó seleccionar un kit de compilación, eligiendo **"Pico ARM GCC"**.

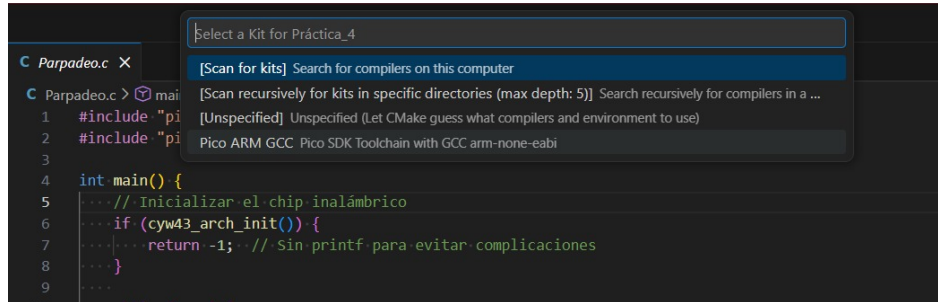


Figura 6: Enter Caption

Figura 7: Selección del kit Pico ARM GCC en VSCode
Espacio para captura de pantalla de la selección del kit

Posteriormente, se utilizó el botón **"Build"** en la barra inferior o en la extensión de CMake para iniciar el proceso de compilación.

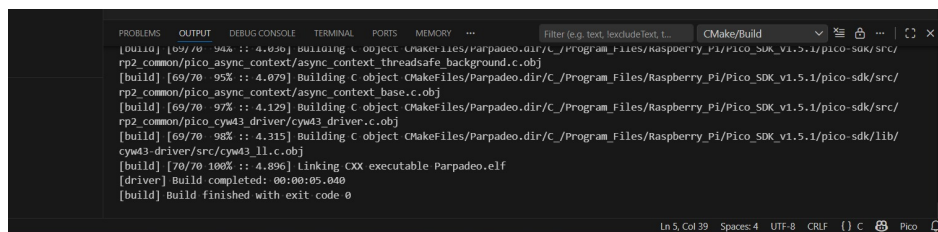


Figura 8: Enter Caption

Figura 9: Proceso de compilación exitoso en VSCode
Espacio para captura de pantalla de la terminal mostrando compilación exitosa

2.3.2. Generación del archivo .uf2

Al finalizar la compilación, se generó una carpeta **build** en el directorio del proyecto. Dentro de esta carpeta se encuentran los archivos generados, incluyendo el archivo **Parpadeo.uf2**, que es el formato necesario para programar la Raspberry Pi Pico W.

2.3.3. Transferencia a la Raspberry Pi Pico W

El proceso de carga del programa a la placa se realizó siguiendo estos pasos:

1. Se desconectó la Raspberry Pi Pico W de la computadora.
2. Se mantuvo presionado el botón **BOOTSEL** en la placa.
3. Sin soltar el botón, se conectó el cable micro USB a la computadora.
4. Se soltó el botón **BOOTSEL**, momento en el cual la placa apareció como una unidad de almacenamiento externa llamada **RPI-RP2**.
5. Se copió el archivo `Parpadeo.uf2` desde la carpeta `build` y se pegó en la unidad **RPI-RP2**.

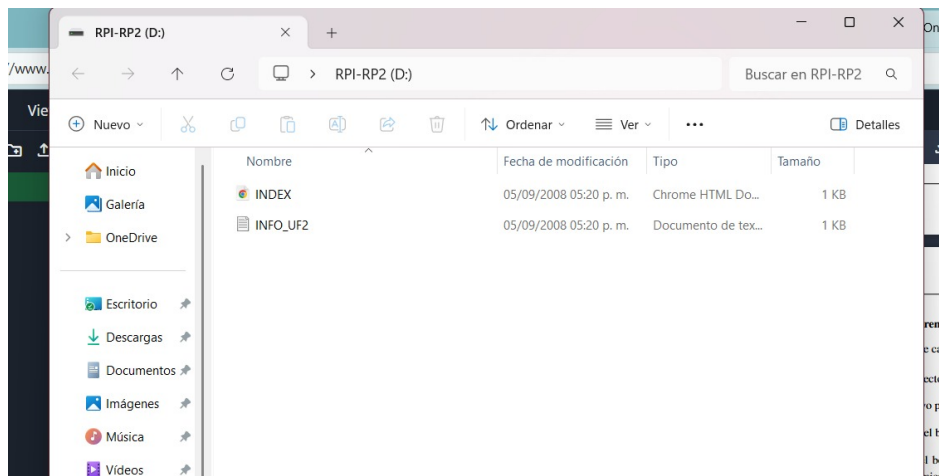


Figura 10: Enter Caption

Figura 11: Unidad RPI-RP2 montada después de presionar BOOTSEL

Espacio para captura de pantalla de la unidad RPI-RP2 en el explorador

2.3.4. Verificación del funcionamiento

Al copiar el archivo `.uf2`, la unidad RPI-RP2 se desconectó automáticamente y la Raspberry Pi Pico W se reinició, comenzando a ejecutar el programa. Inmediatamente, el LED integrado en la placa comenzó a parpadear con una frecuencia de 4 parpadeos por segundo (250ms encendido, 250ms apagado).



Figura 12: Enter Caption

Figura 13: LED integrado de la Pico W parpadeando

Espacio para fotografía del LED de la Pico W encendido durante el parpadeo

3. Resultados

3.1. Código utilizado

El código fuente completo utilizado en la práctica es el presentado en la sección anterior. Este código logra exitosamente el parpadeo del LED integrado aprovechando las librerías del SDK.

3.2. Evidencias del proceso

A lo largo de este documento se han incluido espacios para las capturas de pantalla que documentan cada etapa del proceso:

- Instalación del SDK de Raspberry Pi
- Estructura de carpetas del proyecto
- Navegación en Pico - Developer PowerShell

- Archivos del proyecto en el explorador
- Selección del kit en VSCode
- Compilación exitosa
- Contenido de la carpeta build
- Unidad RPI-RP2 montada
- Fotografía del LED funcionando

3.3. Errores encontrados y soluciones

Durante el desarrollo de la práctica se presentaron los siguientes errores:

- **Error 1: Archivo CMakeLists.txt mal nombrado.** Inicialmente el archivo fue creado como `ÇmkeLists.txt` (con una 'm' en lugar de 'M'), lo que impedía que CMake lo reconociera. Se solucionó renombrando correctamente el archivo a `ÇMakeLists.txt`.
- **Error 2: Archivo Parpadeo.c con doble extensión.** El archivo fuente fue creado como `"Parpadeo.c.c"`, lo que causaba errores de compilación. Se solucionó renombrando a `"Parpadeo.c"`.
- **Error 3: CMake no instalado.** Al intentar compilar en VSCode aparecía el error `"Bad CMake executable"`. Se solucionó descargando e instalando CMake desde la página oficial, asegurándose de marcar la opción `.Add CMake to PATH` durante la instalación.

Estos errores y sus soluciones permitieron comprender la importancia de seguir las convenciones de nomenclatura y de tener correctamente configurado el entorno de desarrollo.

4. Conclusión

4.1. Aprendizaje adquirido

Esta práctica permitió comprender el flujo de trabajo completo para programar la Raspberry Pi Pico W, desde la configuración del entorno con el SDK y CMake, hasta la escritura del código en C y su carga en el microcontrolador. Se aprendió la importancia de seguir las convenciones de nombres y a utilizar las librerías específicas para el hardware de la placa, particularmente el manejo del chip inalámbrico CYW43 para acceder al LED integrado.

La experiencia con los errores iniciales resultó especialmente valiosa, ya que proporcionó un entendimiento más profundo de cómo funciona el sistema de compilación y la importancia de cada archivo en el proyecto.

4.2. Posibles mejoras en la implementación

El código actual podría mejorarse de las siguientes maneras:

- Implementar diferentes patrones de parpadeo (por ejemplo, 3 parpadeos rápidos seguidos de una pausa larga).
- Utilizar interrupciones de temporizador en lugar de retardos bloqueantes para permitir multitarea.
- Añadir control mediante un botón externo para cambiar la frecuencia de parpadeo.
- Implementar una comunicación serial para mostrar mensajes de depuración.

4.3. Aplicaciones prácticas del conocimiento adquirido

Los conocimientos adquiridos en esta práctica son fundamentales para desarrollar proyectos más complejos, tales como:

- Control de iluminación LED para señalización o decoración.
- Lectura de sensores con indicadores visuales de estado.
- Implementación de semáforos o sistemas de alerta.
- Proyectos de IoT donde el LED puede indicar el estado de conexión Wi-Fi.
- Wearables con indicadores luminosos de notificaciones.

La capacidad de crear proyectos desde cero y comprender la estructura del SDK de Raspberry Pi abre la puerta al desarrollo de aplicaciones profesionales en el campo de los sistemas embebidos.

Anexo: Criterios de Evaluación

Criterio	Puntaje
Configuración del Entorno: Documentación detallada de la instalación del SDK y configuración del proyecto	2 pts
Desarrollo del Código: Correcta implementación y explicación del código utilizado	1 pt
Evidencias: Capturas de pantalla o fotografías que demuestren el funcionamiento del microcontrolador	1 pt
Presentación y claridad: Redacción, ortografía y estructura del documento	1 pt
Total	5 pts

Cuadro 1: Criterios de evaluación del reporte técnico